Ubuntu 与 ROS 基础

期末笔试复习资料

知识点 · 题型方法 · 操作流程 · 代码范例 · 练习答案

资料依据：Lesson1-Lesson10，10份PDF，共1330页；结合本对话中可见的填空题、主观题、命令操作与报错。

适用环境：ROS 1 Noetic / Ubuntu 20.04 / Catkin / C++ roscpp。

整理日期：2026年10月11日

本资料按知识链组织，保留课件主要术语，并为笔试补上答题步骤、完整代码与例题。阅读时先理解每章的对照关系，再遮住答案手写命令和程序。

复习重点由课件学习目标、章末 Post-assessment 和已提供习题推断。Lesson9第15页明确标出 C++ 节点编程为“考试编程题”；其他题型与模拟卷为复习训练，不是已取得的正式试卷。

阅读导航

阅读导航 2

01 复习范围、顺序与题型方法 3

02 Linux、Ubuntu、文件系统与FHS 5

03 终端、Shell、TTY与基础命令 6

04 Shell脚本、变量、重定向与管道 8

05 归档压缩、用户组与权限 9

06 软件包管理、APT与PPA 11

07 ROS概述、安装与命令工具 13

08 ROS架构、Catkin、XML与package.xml 14

09 消息、服务与CMake生成规则 17

10 计算图、通信模型、工具与turtlesim 19

11 C++基础、编译、调试与工程 21

12 roslaunch：理解、编写与执行 22

13 ROS C++编程：11步骤与完整例子 25

14 高频错误、题型训练与答题模板 29

附录A 已提供的7组填空题：116空速查 31

附录B 主观题的标准答题要点 34

附录C 自拟期末模拟卷与参考答案 36

附录D 覆盖核对、后续专题与考前清单 39

01 复习范围、顺序与题型方法

1.1 资料覆盖说明

文件夹中实际为10份PDF。Lesson4第27页起复习Lesson1-Lesson4；Lesson10第45页起复习Lesson5-Lesson10。本资料合并重复内容，以每讲新授部分、学习目标和章末题核查重要知识点。页码均指PDF文件页码。

课件	新授内容	本资料对应章节
Lesson1，102页	ROS概述；Unix/GNU/Linux；Ubuntu虚拟机、GNOME	02、07及附录D
Lesson2，79页	文件、FHS；终端、TTY、Shell；基础命令	02、03
Lesson3，75页	Shell变量；重定向、管道；归档压缩；权限	04、05
Lesson4，272页	软件包管理；后半为Ubuntu复习	06
Lesson5，58页	ROS定义与理念；Noetic安装；rosbash	07
Lesson6，87页	ROS架构；工作空间；XML、功能包；消息服务	08、09
Lesson7，84页	CMake；计算图；turtlesim	09、10
Lesson8，82页	C++基本语法、编译调试、类；roslaunch	11、12
Lesson9，65页	话题/服务通信模型；roscpp 11步骤；发布订阅	10、13
Lesson10，426页	自定义消息/服务的C++程序；后半为ROS复习	09、13

课程总纲还列出TF2、URDF、Xacro、Rviz/Gazebo、SLAM、地图、定位、控制和路径规划，但所提供的10份课件未展开相应专题。附录D给出范围提醒与基础概念，不能据这批文件确认后续专题的期末深度。

1.2 建议的三轮复习

第一轮“认清概念”：按02-12章阅读，合上资料后解释每组相近概念，尤其两套ROS三层结构、消息与话题、话题与服务、工作空间与功能包。

第二轮“手写可执行答案”：默写Linux命令、权限换算、tar四条命令、工作空间三条命令、XML、消息/服务定义、CMake顺序和C++骨架。之后对照标点、大小写、路径与参数。

第三轮“限时综合”：完成模拟卷，把错题改写成一句规则，次日再次闭卷完成。编程题应先画数据流，再写消息类型、节点接口和构建配置。

可用时间	先做什么	检验方式
约一周	每天完成2个知识模块，末两天做模拟卷和错题	能独立解释并手写流程
约三天	第一天Linux；第二天Catkin/通信/launch；第三天程序与模拟卷	代码骨架和命令闭卷写出
考前一天	先读14章改错、附录A填空速查；再默写13章核心代码	只复查仍不会的部分

1.3 笔试常见题型的作答方法

题型	方法	常见失分点
填空/选择	根据问法判断是概念、命令、目录还是消息类型；多选逐项核验	凭关键词选择、混淆字段与常量
名词解释	先定义，再写作用，再给一个课程例子	只写英文全称、没有作用
比较题	在同一维度比较对象、通信时序、适用场景	只分别背定义，没有区别
写命令	先确定对象，再选命令与参数，最后写路径	少空格、参数方向错、大小写错
权限题	识别u/g/o，再把r/w/x转4/2/1，分组三位计算	把首字符文件类型算作权限
程序填空	先确定发布/订阅或服务角色，再填API与类型	节点名、话题名、类型混在一起
流程排序	按“定义数据→声明依赖→生成代码→编译节点→运行”	CMake中生成与导出顺序颠倒
改错/报错	找第一条有原因的Error；核对路径、文件、XML、依赖和顺序	只看最后一行make失败

模拟题、自拟例子和补充解释均用于练习。此前提到的40道选择题的完整题干与选项在现有消息中不可见，本资料不编造逐题答案。

02 Linux、Ubuntu、文件系统与FHS

课件依据：Lesson1第33-38、40-59、68-97页；Lesson2第5-26页。

2.1 Unix、GNU、Linux与发行版

Unix是重要的操作系统体系；GNU项目提供大量自由软件工具；Linux是操作系统内核。通常使用的GNU/Linux发行版把内核、系统工具和应用软件集成成可安装的完整系统。Ubuntu基于Debian发展而来。

GNU GPL是General Public License。按课件应掌握运行、复制、研究修改、发布改进版本等自由；它促进Linux的开放开发和共享。自由软件的“自由”主要指权利，并不等同于软件必定免费。

Linux内核版本指内核自身的版本，发行版版本由发行者管理。Ubuntu 20.04不是Linux内核版本号；同一发行版可安装不同内核。

2.2 虚拟机与桌面环境

虚拟机在宿主操作系统中提供虚拟硬件来运行客户操作系统。安装顺序通常为：安装虚拟化软件→创建Linux虚拟机→分配内存/CPU/虚拟磁盘→挂载Ubuntu ISO→安装客户系统→安装客户增强工具→检查显示、网络、共享等。

VirtualBox Extension Pack扩展宿主端的某些功能，如USB支持；Guest Additions安装在客户系统中，用于显示、鼠标集成、共享剪贴板/文件夹等。二者安装位置和作用不同。具体支持能力随版本变化，复习时识别其角色。

Linux常见桌面环境包括GNOME、KDE Plasma、Xfce等。课件使用Ubuntu 20.04的GNOME桌面。应用搜索、文件管理器、系统设置、GNOME扩展属于图形操作内容；笔试宜会说功能与操作目的。打开仿真终端常用Ctrl+Alt+T。GNOME扩展可借助扩展管理工具/仓库包或相应网页扩展机制配置。

2.3 文件、文件名与路径

按课件模型，文件由索引节点（inode）和数据两部分组成。inode保存类型、权限、所有者、大小、时间等元数据以及数据定位信息；文件名由目录项关联inode。不要把inode理解成文件内容本身。

课件归纳四类文件：普通文件、目录文件、设备文件、链接文件。设备可进一步分块设备、字符设备。Linux实际还存在管道、套接字等类型，回答“四种”时按课件分类。

文件名区分大小写，隐藏文件名常以点号开头。扩展名主要是命名约定，不能仅凭扩展名认定实际格式。路径分隔符是/；文件名中不能包含/和NUL。带空格的路径应使用引号。

记号	意义	示例
/	根目录或路径分隔符	/home/thl
~	当前用户家目录	~/catkin_ws
.	当前目录	./demo.sh
..	父目录	cd ..
绝对路径	从根目录开始定位	/home/thl/a.txt
相对路径	从当前工作目录开始定位	../a.txt

例题：当前目录为/home/thl/catkin_ws/src，执行cd ../..后在哪里？解法：第一次..回到catkin_ws，第二次..回到thl，因此结果是/home/thl。

2.4 FHS的两层规范与四种形态

FHS是Filesystem Hierarchy Standard。第一层规定根目录下各主要目录应存放哪些文件；第二层主要细化/usr和/var的子目录。目录形态由“可共享/不可共享”和“静态/可变”组合为四类。“静态”指正常操作中内容不经常改变，不表示永远不变。

目录	主要用途	记忆与考法
/bin	基本用户命令	ls、cp等基本命令；现代系统可能链接至/usr/bin
/sbin	系统管理命令	系统管理相关程序
/etc	系统配置文件	ROS软件源文件也可放在其下
/home	普通用户家目录	/home/thl
/root	root用户家目录	与根目录/不同
/dev	设备文件	硬件以设备接口呈现
/boot	启动相关文件	内核、引导相关内容
/lib	基础库和模块	支持基本程序运行
/usr	多用户应用、库、文档等	usr不等于当前用户家目录
/var	经常变化的数据	日志、缓存等
/tmp	临时文件	不能作为永久保存位置
/opt	附加软件	ROS 1常安装在/opt/ros/noetic
/proc、/sys	系统/内核信息接口	许多条目是虚拟文件
/media、/mnt	挂载点	可移动介质、临时挂载等

简答题“Linux与Windows目录组织有何不同”：Linux把各文件系统挂载到以/为根的统一目录树；Windows常以C:、D:等驱动器标识组织路径。两者都支持分层目录，不能答成“Windows没有树结构”。

03 终端、Shell、TTY与基础命令

课件依据：Lesson2第40-74页；对应已提供的终端填空题和Post-assessment。

3.1 概念与命令格式

终端提供文本输入/输出接口；图形桌面里的仿真终端是窗口程序；TTY可指终端设备/文本控制台接口。Shell是解释并执行命令的程序，也是一种脚本编程环境。Ubuntu常用Bash。关系可写为：用户→终端→Shell解释命令→程序执行→终端显示输出。

提示符thl@ubuntu:~/桌面$通常包含用户名、主机名、当前目录和权限提示符。$常表示普通用户，#常表示root；提示符可自定义，不能仅依显示字符断言实际身份。sudo使获授权用户以另一个身份执行指定命令，默认通常是root。

命令的一般格式为“命令 [选项] [参数]”。例如mkdir -p ~/catkin_ws/src中，mkdir是命令，-p是选项，后面是路径。资料代码框内不包含用于展示的$提示符。

3.2 十条基础命令

命令	基本作用与常用参数	例子
pwd	打印当前工作目录路径	pwd
ls	列出目录内容；-a含隐藏；-l详细	ls -al ~/
cd	切换当前Shell工作目录	cd ~；cd ..；cd -
echo	输出字符串/变量	echo "$HOME"
touch	更新文件时间；不存在则创建空文件	touch a.txt b.txt
mkdir	创建目录；-p创建缺失父目录	mkdir -p text/1/2/3/4
cp	复制；-r递归；-n不覆盖；-i询问	cp -rn /etc /tmp/
mv	移动或改名；-i覆盖前询问	mv old.txt new.txt
rm	删除；-r递归；-i询问	rm -i a.txt
cat	查看/连接文本文件	cat a.txt；cat a b

“打印当前目录清单”答ls；“打印当前目录路径”答pwd。cd空参数回家目录；cd -返回上一次工作目录。cp和mv的目标是否已经是目录，会影响结果名称，做路径题须先判断目标状态。

3.3 通配符与引用

*匹配任意数量字符，?匹配单个字符，[ABC]匹配给定字符之一。普通通配一般不匹配点号开头的隐藏名。大小写敏感，所以A*和a*不同。通配符通常由Shell先展开，再将匹配结果传给命令。

例题：删除/tmp下所有A开头的普通文件。纸面常写rm /tmp/A*；它不会默认递归删除匹配到的目录。实际操作前应先ls /tmp/A*核对对象。若题目明确包括目录，才涉及-r；不要为“文件”题擅加递归删除。

单引号使内容基本按字面处理；双引号允许$变量展开，同时保留字符串内空格。例如echo '$HOME'显示$HOME字面字符，而echo "$HOME"显示家目录路径。

3.4 五道命令题的完整解法

题目	答案	原因
最快回当前账户主目录	cd或cd ~	不需要记/home下用户名
/etc复制到/tmp且不覆盖	cp -rn /etc /tmp/	-r复制目录，-n跳过已有目标文件
一次创建text/1/2/3/4	mkdir -p text/1/2/3/4	建立嵌套父目录；不是创建四个并列目录
创建四个并列目录	mkdir -p text/{1,2,3,4}	Bash花括号展开；与上一题结构不同
删除/tmp下A开头的文件	rm /tmp/A*	路径和大写A均须准确

写命令的检查顺序：对象是文件还是目录→操作是查看/复制/移动/删除→选参数→检查源与目标顺序→检查路径、空格与大小写。

04 Shell脚本、变量、重定向与管道

课件依据：Lesson3第5-23页。

4.1 脚本编写与执行

Shell执行命令分交互式与批处理式。脚本是文本文件，通常由解释器执行。首行#!/bin/bash指定直接执行时使用的解释器；普通以#开头的内容是注释。#!必须是半角字符。

```
#!/bin/bash
name="ROS learner"
echo "Hello, $name"
```

课件列出的三种执行方式是直接执行、指定Shell执行、将脚本内容作为Shell标准输入。下面用demo.sh说明：

```
chmod +x demo.sh
./demo.sh
bash demo.sh
bash < demo.sh
```

直接执行要求执行权限，./表示当前目录；bash demo.sh通常只需脚本可读。source demo.sh或. ./demo.sh在当前Shell执行，因此可改变当前Shell的变量、目录；它与“另启动一个Shell执行脚本”不同。ROS setup.bash要用source加载环境。

4.2 三类变量与环境文件

按课件分类：用户自定义变量、环境变量、系统变量。用户定义name=value；变量名区分大小写，常用字母/下划线开头，后续含字母、数字、下划线；引用$name或${name}；unset name删除变量。赋值等号两侧不能有空格，含空格的值要引用。

```
course="ROS basics"
echo "$course"
export COURSE="$course"
echo "$PATH"
unset course
```

环境变量可被子进程继承。PATH是以冒号分隔的可执行命令搜索目录；加入路径时常保留原值，例如export PATH="$PATH:/home/thl/bin"。不要把所有变量都当成自动导出的环境变量。

课件中的系统变量可联想常用特殊参数：$0为脚本/程序名称，$1等为位置参数，$#为参数数目，$?为上一命令退出状态，$$为当前Shell进程号。此处为便于理解的补充，具体题目按课件问法回答。

~/.bashrc主要配置用户交互式Bash；~/.profile用于用户登录环境；/etc/profile是系统级登录Shell配置；/etc/bash.bashrc是系统级交互式Bash配置。/etc/environment通常是键值环境配置，不应当作Bash脚本随意执行；其中的变量展开规则与Shell脚本不同。

4.3 三种标准数据流

每个进程通常具备标准输入stdin（文件描述符0）、标准输出stdout（1）、标准错误stderr（2）。标准输出与错误输出是两个独立数据流。

写法	意义	示例
命令 < in.txt	输入从文件读	sort < in.txt

写法	意义	示例
命令 > out.txt	标准输出覆盖写	ls > files.txt
命令 >> out.txt	标准输出追加写	echo hello >> log.txt
命令 2> err.txt	标准错误写文件	ls no_such_file 2> err.txt
命令 > all.txt 2>&1	输出与错误指向同一文件	顺序要正确
命令1 管道 命令2	前者标准输出成为后者标准输入	ls与grep组合

```
sort < in.txt > out.txt
ls | grep '.txt'
```

第一条从in.txt读取内容，排序后写out.txt，不会覆盖in.txt。第二条筛选ls的标准输出。管道默认传递stdout，不自动合并stderr。

例题“输出重定向作用”：改变标准输出的目的地，常用于保存程序输出。例题“重定向与管道区别”：重定向改变输入/输出连接的目标；管道连接两个程序的数据流。

05 归档压缩、用户组与权限

课件依据：Lesson3第32-52、61-70页。

5.1 归档与压缩

归档把多个文件/目录组织成一个文件，便于备份、保存、传输，并可保存部分文件属性；压缩通过编码减少数据体积。tar主要用于归档和提取；gzip、bzip2用于压缩；zip同时组织并压缩文件。归档不必然减小体积。

格式	创建/压缩	提取/解压
.tar	tar -cvf a.tar dir/	tar -xvf a.tar
.gz	gzip file	gunzip file.gz或gzip -d
.bz2	bzip2 file	bunzip2 file.bz2或bzip2 -d
.zip	zip -r a.zip dir/	unzip a.zip
.tar.gz	tar -zcvf a.tar.gz dir/	tar -zxvf a.tar.gz
.tar.bz2	tar -jcvf a.tar.bz2 dir/	tar -jxvf a.tar.bz2

tar参数记忆：c=create创建；x=extract提取；t=list查看归档清单；v=verbose显示过程；f=file指定归档名；z=gzip；j=bzip2。f后要跟归档文件名。

```
tar -zcvf course.tar.gz course/
tar -zxvf course.tar.gz
tar -jcvf course.tar.bz2 course/
tar -jxvf course.tar.bz2
tar -tvf course.tar.gz
```

例题“file.tar.gz用什么命令解压”：tar -zxvf file.tar.gz。例题“把src压缩成src.tar.bz2”：tar -jcvf src.tar.bz2 src/。先看后缀选z或j，再选c或x，最后按“归档名、被打包对象”顺序写。

5.2 三类身份与三类权限

身份：所有者u（user）、所属组g（group）、其他用户o（others）；a代表all，用于权限修改时同时指定三类。权限：r读、w写、x执行；数字权重4、2、1。文件可有访问控制扩展，本课程重点是这组三位传统权限。

对象	r	w与x
普通文件	读取内容	w修改内容；x允许作为程序执行
目录	读取目录项清单	w修改目录项；x进入/搜索该目录

删除文件主要涉及父目录权限，不能仅凭文件本身是否有w判断能否删除。实际还会受到粘滞位、ACL等约束，此课程通常考基础权限。

ls -l输出中的首字符表示文件类型：-普通文件，d目录，l符号链接，b块设备，c字符设备；紧接着九位是u/g/o三组权限。随后通常是链接数、所有者、组、大小、时间、名称。

5.3 数字权限的计算方法

每组分别把出现的权重相加，得到三位八进制数；缺失权限用-表示。例如rwx=7，rw-=6，r-x=5，r--=4，---=0。

权限字符串	数字	含义
rw-r--r--	644	所有者读写；组和其他只读
rwxr-xr-x	755	所有者全权限；组和其他读执行
rw-r-----	640	所有者读写；组只读；其他无
r-xr-xr-x	555	三类均读和执行

符号法用“对象＋操作＋权限”：+增加，-删除，=设为精确集合。

```
chmod u+x demo.sh
chmod go-w file
chmod u=rw,go=r file
chmod 644 file
chmod a=rx /home/ros/myfile
chown thl file
chgrp students file
```

chmod修改权限，chown修改所有者（也可同时指定组），chgrp修改所属组。所有者通常可修改自己的文件权限，chmod并非一律需要sudo；改所有者通常需要特权。

例题：与chmod 644 file等价的命令？先拆成u=6=rw、g=4=r、o=4=r，答chmod u=rw,go=r file。例题chmod a=rx含义？答所有类别用户有读和执行权限、无写权限，数值555。

06 软件包管理、APT与PPA

课件依据：Lesson4第5-20页。

6.1 软件包种类与安装方式

Linux软件包按课件分源码包和二进制包。源码包包含源代码，通常需编译；二进制包通常已针对某环境编译。Ubuntu/Debian常用DPKG体系，包格式为deb；RPM体系常见于其他发行版。压缩后缀并不能保证内容一定为源代码。

课件的安装方式包括源码安装、deb安装、bin安装、免安装包、PPA软件源安装。APT/Ubuntu软件中心也可从配置的软件源获取软件。

6.2 源码包的答题流程

步骤：取得包→解压→进入源码目录→阅读README/INSTALL→检查配置→编译→安装→必要时清理构建文件。课件的典型Autotools流程如下，并非所有项目都用configure：

```
tar -zxvf soft.tar.gz
cd soft
./configure
make
sudo make install
make clean
```

./configure检查编译所需环境并生成构建配置；make按构建规则编译；make install按规则安装；make clean清理构建产物。安装步骤要求权限时才使用sudo，配置/编译通常可用普通用户。

可执行bin包一般先chmod +x soft.bin，再./soft.bin；免安装包一般解压后直接运行其可执行文件。注意./soft.bin与/soft.bin定位的路径不同。

6.3 APT与dpkg

命令	作用	容易混淆的地方
sudo apt update	更新本地软件包索引	不直接升级所有已装软件
sudo apt upgrade	升级已安装软件包	与更新索引是两步
sudo apt install 包名	获取并安装软件及依赖	包名不一定等于PPA名
sudo apt remove 包名	删除软件，通常保留配置	purge还删除包管理的配置
sudo apt purge 包名	删除软件及其受管理配置	用户目录内容不必都被删除
sudo apt autoremove	删除不再需要的自动安装依赖	不是清空所有安装备份
sudo apt clean	清理已下载包缓存	不卸载正在使用的软件
sudo apt autoclean	清理不可再获取等旧包缓存	范围通常小于clean
apt search 关键词	搜索软件包	查询一般无需sudo
sudo dpkg -i soft.deb	安装本地deb	不自动下载补全依赖
dpkg -l	查看已安装包信息	与-i安装不同
sudo dpkg -r 包名	删除包，保留配置	dpkg -e是提取控制信息，不是卸载

APT建立在底层包管理工具之上，可从仓库取得软件并解决依赖；dpkg直接管理deb包。dist-upgrade/full-upgrade主要允许为完成包升级调整依赖，不应简单解释为“把Ubuntu发行版升级到下一版本”。

6.4 软件源与PPA

PPA全称Personal Package Archive，是个人/团队的软件仓库机制。简写形式ppa:所有者/归档名，如ppa:user/ppa-name。课件顺序为添加源→更新索引→安装目标软件包。

```
sudo add-apt-repository ppa:user/ppa-name
sudo apt update
sudo apt install actual-package-name
```

actual-package-name是示意的目标软件包名称；PPA归档名不保证就是包名。Ubuntu 20.04的传统源可写在/etc/apt/sources.list以及/etc/apt/sources.list.d/*.list。

传统源行示例：deb http://example.org/ubuntu focal main。按实际语法应理解为类型、URI、发行套件、组件；课件第19-20页把后两项合并讲为“发行版本或分类”，归纳成三个字段。按课件填空可答：软件包类型、源URL、发行版本或分类；解释真实源行时应分清suite与components。

组件main、universe、restricted、multiverse分别涉及官方支持开源、社区维护开源、受限/专有驱动等、法律/授权受限软件。focal是Ubuntu 20.04代号，不是软件包名。

07 ROS概述、安装与命令工具

课件依据：Lesson1第12-13页；Lesson5第10-25、34-56页。

7.1 定义、目标、四位一体

ROS是Robot Operating System。课件称其为适用于机器人的元操作系统，提供硬件抽象、设备控制、功能实现、进程间通信、软件包管理等能力；通常运行于宿主操作系统之上。它强调模块化、分布式和代码复用。

首要设计目标：提高机器人研发领域的代码复用率。与此对应，项目可将不同机器人可复用的算法、驱动、通信与工具封装为功能包。

英文词	课件术语	理解与例子
Plumbing	通讯机制	节点之间的数据连接、话题和服务
Tools	开发工具/工具软件包	调试、可视化、记录等工具
Capabilities	应用功能/机器人高层技能	可复用的机器人功能与算法
Ecosystem	机器人生态系统	社区、文档、软件包和开发者

ROS历史、开源理念与发展可作识记内容：ROS 1与ROS 2架构不同。本资料中的roscore、rosrun、Catkin、roscpp用法属于ROS 1。Noetic配套Ubuntu 20.04；不能将ROS 2的colcon或ros2 run混入本课程命令题。

课件列出的五个特点为点对点设计、多语言支持、架构精简且集成度高、组件化工具丰富、免费且开源。理解时分别联系节点分布式通信、语言无关的数据接口、可复用模块、Rviz/Gazebo/rosbag/rqt，以及开放社区。具体包许可证可不同，不应说所有ROS生态软件统一受同一许可证约束。

学习ROS五个阶段（Lesson5第24页图）：基础准备→常见功能包→仿真实操→机器人跑起来→聚焦研究重心。前期先补Ubuntu与编程基础，随后学习launch/TF等功能包，经过仿真和真实机器人实践，再面向SLAM、导航等方向深入。该图是学习路线，不是五种ROS软件层次。

7.2 Noetic安装七步与作用

课件步骤	做什么	为什么
1	配置Ubuntu仓库及镜像	取得系统包与依赖
2	添加ROS软件源	让APT找到ROS发行版包
3	配置签名密钥	验证仓库发布的信息
4	更新索引并安装Noetic	desktop-full、desktop、ros-base等
5	加载ROS环境	让命令和包路径可发现
6	安装开发工具、初始化/更新rosdep	准备构建并解析系统依赖
7	启动与测试	roscore、turtlesim等

ros-latest.list记录ROS APT仓库信息；rosdep用于根据功能包依赖声明解析/安装系统依赖。sudo rosdep init通常初始化一次；rosdep update更新依赖数据库，一般用普通用户运行。rosdep失败时应检查具体网络/源/索引错误，不能将所有错误归结为包代码错误。

```
sudo apt update
sudo apt install ros-noetic-turtlesim
source /opt/ros/noetic/setup.bash
roscore
```

安装A软件包一般写sudo apt install ros-noetic-A，其中A须是实际APT包名；ROS包的下划线常在Debian包名中转为连字符。加载环境可写入~/.bashrc，但要避免重复追加。课件的apt-key、旧镜像地址等是历史安装示例；笔试重在步骤和作用，实际安装需核对当前仓库状态。

7.3 rosbash与包依赖命令

工具	作用	典型格式
rosrun	运行指定功能包中的可执行程序	rosrun 包名 程序名
roscd	跳转功能包目录	roscd turtlesim
rosls	列出功能包内容	rosls turtlesim
rosed	编辑功能包中的文件	rosed 包名 文件名
roscp	复制功能包中的文件	roscp 包名 文件名 目标路径
rospack find	查找包的目录	rospack find turtlesim
rospack depends1	查看直接依赖	rospack depends1 beginner_tutorials
rospack depends	查看包含间接依赖的依赖关系	rospack depends beginner_tutorials

课件列出的turtlesim四个可执行程序是turtlesim_node、turtle_teleop_key、draw_square、mimic。终端里应使用turtle_teleop_key，课件安装测试页的turtlesim_teleop_key是笔误。

08 ROS架构、Catkin、XML与package.xml

课件依据：Lesson6第5-22、30-57页。

8.1 两套三层结构

ROS系统架构：系统层、中间层、应用层。ROS自身结构的三个观察层次：文件系统级、计算图级、开源社区级。前者描述软件体系分层，后者分别从磁盘组织、运行通信和社区资源观察ROS。

文件系统级关注工作空间、功能包、消息/服务描述和构建文件；计算图级关注节点、Master、参数服务器、话题、服务、消息、bag；社区级关注发行版、仓库、文档、包维护和交流。

8.2 创建工作空间、包与加载环境

目录	内容	复习判断
src	功能包与源文件	手工编写的代码主要在这里
build	构建中间内容与CMake缓存	通常不要当源代码改
devel	开发空间结果、库、生成头文件、环境脚本	编译后source devel/setup.bash
install	可选安装空间	catkin_make install时产生安装内容

```
source /opt/ros/noetic/setup.bash
mkdir -p ~/catkin_ws/src
cd ~/catkin_ws
catkin_make
cd ~/catkin_ws/src
catkin_create_pkg robot_course_pkg roscpp rospy std_msgs
cd ~/catkin_ws
catkin_make
source ~/catkin_ws/devel/setup.bash
```

“创建工作空间三条命令”只要求mkdir -p、cd、catkin_make。创建包要在src下；编译要回工作空间根。新终端需要加载环境，源码包名是robot_course_pkg，工作空间名catkin_ws和包名是两个层级。

8.3 功能包结构与文件作用

最小Catkin包需要自己的目录、符合规范的package.xml、Catkin版CMakeLists.txt。一个包目录内不应再嵌套另一个包。

路径/文件	作用
package.xml	包名、版本、描述、维护者、许可证、依赖等清单
CMakeLists.txt	配置构建、生成消息、编译与链接目标、导出构建信息
src/	C/C++等源文件
include/	通常存头文件
scripts/	常放Python等可执行脚本
msg/	消息类型描述.msg
srv/	服务类型描述.srv
launch/	启动配置.launch

工作空间src/CMakeLists.txt是顶层构建入口，与src/robot_course_pkg/CMakeLists.txt不同。修改自定义消息生成规则时编辑后者。

8.4 XML的基本结构与改错

XML是可扩展标记语言。声明示例<?xml version="1.0" encoding="UTF-8"?>包含版本和编码。文档可包含元素、属性、注释等；声明不是根元素。元素可成对或自闭合；整个文档只有一个根元素；标签区分大小写，嵌套不能交叉；属性值要加引号。

```
<?xml version="1.0" encoding="UTF-8"?>
<Student id="S100">
  <Name>Tom</Name>
</Student>
```

注释形式<!-- comment -->；&、<等出现在普通文本/属性值中时需按规则转义，例如&、<；CDATA可容纳不按普通标签解析的文本。命名空间机制允许特定冒号用法，因此“XML名称绝不允许冒号”并非普遍规则；本课程的普通自定义标签无需使用它。

8.5 package.xml必备项与六种依赖

课件重点基本项：name、version、description、maintainer、license。依赖不是把包安装进去，而是声明构建、运行、测试等各阶段需要什么。

标签	意义
buildtool_depend	构建工具依赖，例如catkin
build_depend	构建时依赖，例如message_generation
build_export_depend	下游包使用本包导出内容进行构建时需要的依赖
exec_depend	运行时依赖，例如message_runtime
test_depend	测试依赖
doc_depend	文档构建依赖

格式2中depend是build_depend、build_export_depend、exec_depend的简写。题目问“六个”时按上表写；不把depend强行作为第七个答案混进去。

```
<?xml version="1.0"?>
<package format="2">
  <name>robot_course_pkg</name>
  <version>0.0.1</version>
  <description>ROS course examples</description>
  <maintainer email="thl@example.com">thl</maintainer>
  <license>BSD</license>
  <buildtool_depend>catkin</buildtool_depend>
  <depend>roscpp</depend>
  <depend>std_msgs</depend>
  <build_depend>message_generation</build_depend>
  <exec_depend>message_runtime</exec_depend>
</package>
```

这是后文示例包所需的简洁清单；不要把新增依赖写到</package>后面。XML能解析不代表依赖一定齐全，还要核对程序用到的包。

09 消息、服务与CMake生成规则

课件依据：Lesson6第65-83页；Lesson7第5-31页；Lesson10第12-18、28-33页。

9.1 消息描述：字段与常量

ROS消息描述可以包含字段Fields和常量Constants。字段写“类型 名称”，代表每个消息实例携带的数据；常量写“类型 常量名=值”，代表固定定义值，不像普通字段那样为每条消息携带可变数据。

另一个分类是标准消息类型与自定义消息类型。题目问“消息包括哪两种”，要结合上下文：后面紧接字段/常量时一般答Fields和Constants；讨论类型来源时答标准与自定义，不能只看“两种”就机械选择。

```
uint8 STOP=0
uint8 RUN=1
string name
uint16 age
float64 height
```

常用基本类型含bool、int8/16/32/64、uint8/16/32/64、float32/64、string、time、duration；可包含其他消息类型。变长数组用type[]，定长数组用type[N]。Header通常携带时间戳、坐标帧信息等。自定义文件置于包的msg目录。

消息类型在命令行写包名/类型名，例如std_msgs/String；C++写std_msgs::String；包含头文件写std_msgs/String.h。三种表示方式必须分别记忆。

9.2 服务描述：请求与响应

.srv放在包的srv目录；请求在---前，响应在---后，两侧使用消息描述字段语法。可以含其他消息类型，不能将一个.srv直接嵌套成另一个服务字段。

```
int32 num1
int32 num2
---
int32 sum
```

这个sum.srv定义客户端提交两个整数，服务端返回它们的和。服务类型可写robot_course_pkg/sum；运行中的服务名可叫/add_two_ints。类型名称和运行服务名称不同。

rosmsg show机器人包/消息类型查看消息定义；rossrv show包/服务类型查看服务定义；rosservice type /服务名查询该运行服务的类型。

9.3 自定义消息与服务的构建顺序

主链：创建.msg/.srv→package.xml声明依赖→find_package查依赖→add_message_files/add_service_files列定义→generate_messages生成→catkin_package导出→add_executable编译节点→add_dependencies保证生成顺序→target_link_libraries链接→catkin_make→source环境。

message_generation用于构建期间生成语言接口；message_runtime用于生成接口的运行/使用。对于本课程使用std_msgs的示例应声明std_msgs依赖；只使用基本字段的类型不代表普遍必须依赖std_msgs。

9.4 可用于后文六个节点的CMake配置

该配置对应13章的talker.cpp、listener.cpp、person_pub.cpp、person_sub.cpp、sum_server.cpp、sum_client.cpp，以及msg/person.msg、srv/sum.srv。必须先实际创建这些文件。

```
cmake_minimum_required(VERSION 3.0.2)
project(robot_course_pkg)

find_package(catkin REQUIRED COMPONENTS
  roscpp
  std_msgs
  message_generation
)

add_message_files(FILES person.msg)
add_service_files(FILES sum.srv)
generate_messages(DEPENDENCIES std_msgs)

catkin_package(
  CATKIN_DEPENDS roscpp std_msgs message_runtime
)
include_directories(${catkin_INCLUDE_DIRS})

foreach(node talker listener person_pub person_sub sum_server sum_client)
  add_executable(${node} src/${node}.cpp)
  add_dependencies(${node}
    ${${PROJECT_NAME}_EXPORTED_TARGETS}
    ${catkin_EXPORTED_TARGETS})
  target_link_libraries(${node} ${catkin_LIBRARIES})
endforeach()
```

如果笔试只编一个节点，写成add_executable(talker src/talker.cpp)、target_link_libraries(talker ${catkin_LIBRARIES})即可表达目标和链接关系；使用自定义消息时还需生成目标依赖。不要只写编译C++而忘记先生成消息头文件。

9.5 各CMake命令的考法

命令	意义
cmake_minimum_required	最低CMake版本要求
project	工程/包名称；应与package.xml一致
find_package	查找构建所需包与组件
catkin_python_setup	使用Catkin的Python模块安装支持；非每包必需
add_message_files / add_service_files	列出.msg/.srv描述文件
generate_messages	触发语言接口生成；必须在catkin_package之前
catkin_package	导出供其他包使用的构建信息
include_directories	添加头文件搜索路径
add_library / add_executable	创建库目标/可执行目标
add_dependencies	规定目标之间构建顺序
target_link_libraries	为目标链接所需库

生成的C++头文件通常在devel/include/包名/消息名.h。编译成功后新终端仍要source devel/setup.bash才能发现当前工作空间的包与生成类型。

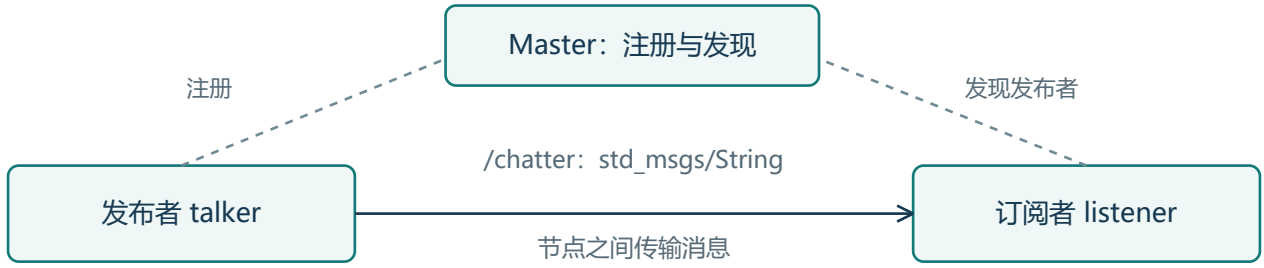
10 计算图、通信模型、工具与turtlesim

课件依据：Lesson7第37-81页；Lesson9第5-11页。

10.1 计算图的七类对象

对象	意义与作用
Node节点	执行计算任务的运行程序/单元；本课常按一个节点进程理解
Master	名称注册与发现，帮助节点找到通信对方
Parameter Server	参数存储与检索；配置数据，不是高频传感器流
Topic话题	按名称发布/订阅消息的通信通道
Message消息	通信传输的数据结构与实例
Service服务	命名的请求/响应接口
Bag	用于记录/回放话题消息的数据文件

Master是管理与发现者，连接建立后话题数据通常直接在发布者与订阅者之间传输。话题图画出的Topic不是必须存在的独立中转程序。一个节点可同时发布/订阅多个话题，也可同时是服务端或客户端。



话题是逻辑通道；图中的Master不转发这条业务数据流。

10.2 发布/订阅的连接流程

步骤：发布者向Master注册话题与自身地址→订阅者向Master注册/查询对应话题→Master帮助发现发布者→订阅者与发布者协商连接→数据通过约定传输方式直接传递。

Topic可有多个发布者和多个订阅者；每个具体话题需要匹配的消息类型。“异步”指发布者不要求每次发布后等待业务响应；不是数据永远无序或完全没有通信协议确认。

10.3 服务请求/响应流程

服务端向Master注册服务名与地址→客户端查找服务→客户端连接服务端并发送请求→服务端处理并回传响应→客户端获得结果。普通同步call会等待结果。服务端应已可用，客户端可waitForExistence等待。

比较维度	话题Topic	服务Service
模式	发布/订阅，异步数据流	请求/响应，通常同步等待
数据结构	一个消息类型	请求与响应两部分

比较维度	话题Topic	服务Service
使用场景	连续速度、位姿、传感器数据	一次清屏、生成对象、求和
接口	advertise/publish、subscribe	advertiseService、serviceClient/call
文件	.msg	.srv

例题看图方法：先找谁发布、谁订阅；沿箭头读消息方向；把Master注册箭头与业务数据箭头分开；再看是否有成对请求/响应。不能凭“图上两个方向都有箭头”就认定所有通信都是服务。

10.4 命令工具按对象配对

对象	命令	用法举例
节点	rosnode list/info/kill	rosnode info /turtlesim
话题	rostopic list/type/echo/info/hz/pub	rostopic type /turtle1/cmd_vel
消息类型	rosmsg show/info/list	rosmsg show geometry_msgs/Twist
服务实例	rosservice list/type/call	rosservice call /clear
服务类型	rossrv show/list	rossrv show turtlesim/Spawn
参数	rosparam list/get/set/delete/load/dump	rosparam get /turtlesim/background_r
图与GUI工具	rqt_graph、rqt	查看节点/话题连接
记录回放	rosbag record/info/play	rosbag record /turtle1/pose

rosbag的记录/回放可供离线分析和重复测试；回放话题数据不能自动重建所有节点内部状态。rostopic hz测观测到的消息频率，不能直接等同代码里期望的Rate。

记录流程：启动数据源→选择话题记录→Ctrl+C正常结束→查看bag信息→需要时启动ROS基础环境回放。-O指定输出文件名，-a可记录所有话题，但应按题意选范围。

```
rosbag record -O turtle_demo /turtle1/pose
rosbag info turtle_demo.bag
rosbag play turtle_demo.bag
rosparam set /turtlesim/background_r 255
rosparam get /turtlesim/background_r
rosparam dump turtle_params.yaml /turtlesim
```

rosparam dump保存参数，load可加载；bag保存消息流，二者不是同一种内容。turtlesim背景参数修改后可调用/clear观察背景刷新。这些步骤是工具用途练习，不是要求考试前改动系统配置。

10.5 小乌龟完整练习流程

分别打开终端，并在每个终端加载ROS环境。第一个启动roscore，第二个启动仿真，第三个运行键盘控制并让该终端获得输入焦点。

```
roscore
rosrun turtlesim turtlesim_node
rosrun turtlesim turtle_teleop_key
```

查看速度话题及消息结构：

```
rostopic echo /turtle1/cmd_vel
rostopic type /turtle1/cmd_vel
rosmsg show geometry_msgs/Twist
```

geometry_msgs/Twist含linear、angular，二者都是Vector3（x、y、z）。turtlesim平面运动主要使用linear.x和angular.z：正linear.x前进，负linear.x后退；angular.z控制转向。

```
rostopic pub -r 1 /turtle1/cmd_vel geometry_msgs/Twist \
  '[1,0,0]' '[0,0,1]'
```

该命令以1 Hz连续发布速度指令，1是发布频率，不是线速度；线速度由消息linear.x=1决定。Ctrl+C终止持续发布。/clear清除轨迹而不重置全部仿真状态；/reset重置仿真，含对象/位置等。

```
rosservice call /clear
rosservice call /spawn 2 2 0 turtle2
rosrun turtlesim mimic input:=turtle1 output:=turtle2
```

mimic的输入为/turtle1/pose，输出为/turtle2/cmd_vel。它读取第一只的速度信息并用于控制第二只，不会保证两只瞬间处于同一位置。第一只后退、第二只也后退是同号线速度复制的正常结果。若第二只名为turtle_mimic，应将output映射到turtle_mimic，不能同时写成turtle2。

11 C++基础、编译、调试与工程

课件依据：Lesson8第5-50页。

11.1 编译和运行

C++源文件通常为.cpp，头文件常为.h。main是程序入口；语句常以分号结束；块由花括号组成。#include包含头文件；命名空间名::成员访问限定名称。

```
#include <iostream>
int main() {
    int a = 3, b = 4;
    std::cout << a + b << std::endl;
    return 0;
}
```

```
g++ main.cpp -o main
./main
g++ -g main.cpp -o main
gdb ./main
```

g++编译并链接C++；-o指定输出文件；-g加入调试信息。gcc是GNU编译器工具的一部分，编译C++时使用g++通常会正确链接C++标准库。GDB常用break设断点、run运行、next步过、step步入、print查看值、continue继续、quit退出。

11.2 类型、函数、指针与引用

基础题注意整型/浮点型、布尔、字符串、if/for/while、函数参数与返回值。整数除法例如7/2为3；7.0/2为3.5。argc含程序本身在内的命令行参数个数，argv保存各参数字符串；运行./demo 3 4时argc为3。

&可表示取地址或引用声明；*可表示指针声明或解引用；指针访问成员用->，普通对象用.。const std_msgs::String::ConstPtr&是对消息常量智能指针的引用，回调中用msg->data取字段。std::string的c_str()用于获得日志%s可使用的C字符串。

11.3 类、对象与访问控制

类定义一组数据与操作，对象是类的实例。class默认成员访问为private，struct默认public；二者在C++中都能有成员函数。public允许外部访问，private限制类内及友元，protected还允许派生类成员访问。

```
#include <iostream>
class Robot {
public:
    int id;
    void move() { std::cout << "move" << std::endl; }
private:
    int secret;
};
class MobileRobot : public Robot {
public:
    void turn() { std::cout << "turn" << std::endl; }
};
int main() {
    MobileRobot r;
    r.id = 2;
    r.move();
    r.turn();
    return 0;
}
```

派生类的成员函数可使用基类public/protected内容，但不能直接使用基类private数据；外部代码也不能访问protected。类定义结束要有分号。课件中的OPP应理解为OOP（面向对象编程）的拼写笔误。

11.4 命名空间与多文件工程

namespace避免同名冲突；robot::process与machine::process可指向两个不同命名空间的函数。头文件放声明，.cpp放定义，main.cpp使用它们；函数的类外定义常写返回类型 类名::函数名(...)。

```
g++ add.cpp main.cpp -o main
./main
```

Makefile规则基本形态为目标:依赖，下一行是命令，命令行一般以Tab开头。make按依赖与时间判断哪些目标需重建。CMake生成构建规则，make执行规则，Catkin组织ROS构建；三者层级不同。

程序阅读题按“变量初值→控制分支→函数参数→返回值→输出顺序”追踪。看到编译错误先核查括号、分号、类型、声明与包含文件，不把运行时通信问题和C++语法错误混在一起。

12 roslaunch：理解、编写与执行

课件依据：Lesson8第56-79页；已提供的node标签、mimic重映射图。

12.1 基本格式与核心属性

launch文件是XML启动配置，可一次组织多个节点、参数、命名空间与映射。根标签是launch。使用roslaunch 包名 文件.launch；必要时roslaunch可自动启动本机Master。rosrun通常只运行一个指定可执行程序，不读取这套XML组织配置。

```
<launch>
  <node pkg="turtlesim" type="turtlesim_node"
        name="sim" output="screen" />
</launch>
```

node属性	含义
pkg	可执行程序所在功能包
type	要运行的可执行程序名称，不是消息类型
name	运行时节点名称
args	传给该程序的命令行参数
output="screen"	将输出显示在终端
respawn="true"	节点退出后自动尝试重启
respawn_delay	重启前等待时间
required="true"	该节点结束会导致整个launch关闭；不与respawn同时设true
ns	节点命名空间
machine	指定运行机器；用于分布式配置

课件例子listener1、rospy_tutorials、listener.py、--test、respawn=true分别表示节点名、包、程序、程序参数、退出后重启。name和type不必相同。

12.2 常见XML标签

标签	作用与理解
launch	整个文件的根元素
node	启动节点
remap	将程序使用的ROS名称映射为实际名称
param	设置一个参数；node内通常为私有参数
rosparam	用YAML等加载/设置一组参数
arg	定义launch参数，用$(arg 名称)引用
include	引入另一个launch文件
group	成组设置命名空间、映射等
env	设置节点进程环境变量
machine	配置远程运行机器
test	在rostest中组织测试节点

param与arg不同：param是运行时参数服务器数据；arg是launch解析时的输入。env是操作系统环境变量，也与参数服务器不同。包路径可用$(find 包名)取得。

12.3 完整例子：话题映射与可变频率

下面的launch对应13章中的talker/listener。talker读取私有参数rate；listener原订阅chatter，经映射改为hello；talker也发布到hello，消息类型保持一致。

```
<launch>
  <arg name="rate" default="10" />
  <node pkg="robot_course_pkg" type="talker"
        name="talker" output="screen">
    <param name="rate" value="$(arg rate)" type="double" />
    <remap from="chatter" to="hello" />
  </node>
  <node pkg="robot_course_pkg" type="listener"
        name="listener" output="screen">
    <remap from="chatter" to="hello" />
  </node>
</launch>

roslaunch robot_course_pkg demo.launch rate:=5
```

步骤：在包下建立launch目录→保存demo.launch→确认type对应的可执行程序已编译→source工作空间环境→运行上述命令。Ctrl+C通常终止该launch所管理的进程。

常见题：“把订阅chatter的程序接到hello发布者”。答在该订阅节点下写<remap from="chatter" to="hello"/>。from是程序原使用名，to是要接入的实际名，方向不能倒。

13 ROS C++编程：11步骤与完整例子

课件依据：Lesson9第15-32、44-62页；Lesson10第14-40页。以下为按正确API整理的参考代码，统一使用robot_course_pkg包，与08-09章配置配套。

13.1 11个知识步骤与API

步骤	作用	API/例子
1	包含ROS与消息头文件	ros/ros.h、std_msgs/String.h
2	初始化节点	ros::init(argc, argv, "talker")
3	输出日志	ROS_INFO/WARN/DEBUG/ERROR/FATAL
4	创建句柄	ros::NodeHandle nh
5	创建消息实例、赋字段	std_msgs::String msg; msg.data=...
6	建立发布者并发送	advertise<类型>、publish
7	注册订阅	subscribe(话题,队列,回调)
8	编写回调	读取消息并处理
9	处理回调	spin或spinOnce
10	控制循环频率	ros::Rate与sleep
11	读取/设置参数	getParam、param、setParam

11项是知识清单，不要求每个程序都同时发布、订阅、读参数。先ros::init再创建NodeHandle；建立通信对象后保持其生命周期；订阅程序通常用spin等待回调；循环里要处理回调可用spinOnce。纯发布者没有回调时不必强行spinOnce。

例1：标准字符串发布者talker.cpp

```
#include <ros/ros.h>
#include <std_msgs/String.h>

int main(int argc, char** argv) {
    ros::init(argc, argv, "talker");
    ros::NodeHandle nh;
    ros::NodeHandle pnh("~");
    double hz;
    pnh.param("rate", hz, 10.0);
    if (hz <= 0) {
        ROS_ERROR("rate must be positive");
        return 1;
    }
    ros::Publisher pub =
        nh.advertise<std_msgs::String>("chatter", 1000);
    ros::Rate rate(hz);
    while (ros::ok()) {
        std_msgs::String msg;
        msg.data = "hello ROS";
        pub.publish(msg);
        ROS_INFO("%s", msg.data.c_str());
        ros::spinOnce();
        rate.sleep();
    }
    return 0;
}
```

评分点：头文件和正确消息类型；节点初始化；句柄；advertise话题和队列；消息赋值；publish；ros::ok循环和Rate。1000是消息队列容量，10 Hz是期望循环频率，二者不是一个概念。

例2：标准字符串订阅者listener.cpp

```
#include <ros/ros.h>
#include <std_msgs/String.h>

void callback(const std_msgs::String::ConstPtr& msg) {
    ROS_INFO("I heard: %s", msg->data.c_str());
}

int main(int argc, char** argv) {
    ros::init(argc, argv, "listener");
    ros::NodeHandle nh;
    ros::Subscriber sub = nh.subscribe("chatter", 1000, callback);
    ros::spin();
    return 0;
}
```

回调参数类型确定收到的消息类型；subscribe可以从回调签名推导类型，也可显式写模板参数。没有spin/spinOnce等回调处理机制时，程序可能立即结束或不处理收到的消息。

运行与观察

在各终端source环境，启动roscore，再分别运行发布者和订阅者。

```
rosrun robot_course_pkg talker
rosrun robot_course_pkg listener
rostopic echo /chatter
rostopic type /chatter
```

若使用12章launch，话题映射为/hello，观察时应改为rostopic echo /hello。ROS日志输出不是发布消息的替代，ROS_INFO只打印日志，publish才发送话题消息。

13.2 自定义消息person的完整实现

课件第13页需求中写float32/int16，而第14页最终person.msg写uint16/float64。这里采用实际定义文件页的类型；考试题干若指定其他类型，应按题干，并同步修改程序。

在msg/person.msg写入：

```
string name
uint16 age
float64 height
```

配置按08章package.xml、09章CMake；编译生成robot_course_pkg/person.h。person是消息类型，person_pub是可执行目标，/person_info是运行话题，这三者可以不同名。

例3：person_pub.cpp，1 Hz发布

```
#include <ros/ros.h>
#include <robot_course_pkg/person.h>

int main(int argc, char** argv) {
    ros::init(argc, argv, "person_pub");
    ros::NodeHandle nh;
    ros::Publisher pub =
        nh.advertise<robot_course_pkg::person>("person_info", 10);
    robot_course_pkg::person msg;
    msg.name = "thl";
    msg.age = 20;
    msg.height = 1.75;
    ros::Rate rate(1);
    while (ros::ok()) {
        pub.publish(msg);
        ros::spinOnce();
        rate.sleep();
    }
    return 0;
}
```

例4：person_sub.cpp

```
#include <ros/ros.h>
#include <robot_course_pkg/person.h>

void callback(const robot_course_pkg::person::ConstPtr& msg) {
    ROS_INFO("name=%s age=%u height=%.2f",
             msg->name.c_str(),
             static_cast<unsigned int>(msg->age), msg->height);
}

int main(int argc, char** argv) {
    ros::init(argc, argv, "person_sub");
    ros::NodeHandle nh;
    ros::Subscriber sub = nh.subscribe("person_info", 10, callback);
    ros::spin();
    return 0;
}
```

复习迁移方法：把std_msgs/String替换为自定义类型时，头文件、C++类型、字段名和构建依赖都需同步改变。只改msg.data为msg.name而保留String类型会编译失败。

13.3 自定义求和服务的完整实现

在srv/sum.srv写两个int32请求与一个int32响应（见09章）。服务节点名sum_server，服务运行名称/add_two_ints，服务类型robot_course_pkg/sum。

例5：sum_server.cpp

```
#include <ros/ros.h>
#include <robot_course_pkg/sum.h>

bool add(robot_course_pkg::sum::Request& req,
         robot_course_pkg::sum::Response& res) {
    res.sum = req.num1 + req.num2;
    ROS_INFO("%d + %d = %d", req.num1, req.num2, res.sum);
    return true;
}

int main(int argc, char** argv) {
    ros::init(argc, argv, "sum_server");
    ros::NodeHandle nh;
    ros::ServiceServer server =
        nh.advertiseService("add_two_ints", add);
    ros::spin();
    return 0;
}
```

回调参数分别是请求与响应的引用；返回true表示本次服务调用处理成功，res.sum才是业务结果。此例只用于小整数求和；超出int32范围的输入不在例题设定内。

例6：sum_client.cpp

```
#include <ros/ros.h>
#include <robot_course_pkg/sum.h>

int main(int argc, char** argv) {
    ros::init(argc, argv, "sum_client");
    ros::NodeHandle nh;
    ros::ServiceClient client =
        nh.serviceClient<robot_course_pkg::sum>("add_two_ints");
    if (!client.waitForExistence(ros::Duration(5.0))) {
        ROS_ERROR("service unavailable");
        return 1;
    }
    robot_course_pkg::sum srv;
    srv.request.num1 = 3;
    srv.request.num2 = 4;
    if (client.call(srv)) {
        ROS_INFO("sum=%d", srv.response.sum);
    } else {
        ROS_ERROR("service call failed");
        return 1;
    }
    return 0;
}
```

client.call返回调用是否成功；srv.response.sum保存响应。客户端完成一次请求后可以结束，不必像订阅者一样永久spin。

```
cd ~/catkin_ws
catkin_make
source devel/setup.bash
rosrun robot_course_pkg sum_server
```

另一个加载环境的终端运行rosrun robot_course_pkg sum_client。也可用命令行调用服务：

```
rosservice call /add_two_ints "num1: 3
num2: 4"
```

13.4 编程题的七步检查

1. 确定角色：发布者、订阅者、服务端或客户端。2. 确定数据类型与字段。3. 包含正确头文件。4. 初始化节点和句柄。5. 建立通信接口并实现数据/回调。6. 保持必要运行循环或等待服务。7. 说明package.xml、CMake、编译与source。

如果题目只要求核心代码，可省略次要日志，但不能省略类型、消息赋值、调用接口和必要回调处理。不要写出与题意无关的所有11项。

14 高频错误、题型训练与答题模板

14.1 你已遇到的错误如何定位

现象	原因线索	修正方法
mkdir-p~/...不存在	把命令、选项、路径连成一个词	mkdir -p ~/catkin_ws/src
package.xml junk after document element	根元素结束后出现多余元素等	所有依赖置于package根内部；核对报错行
test.msg does not exist	CMake列的文件未创建/名称不一致	核对包/msg/test.msg与大小写
generate_messages须先调用	CMake顺序颠倒	先生成消息，再catkin_package
顶层只见CATKIN_TOPLEVEL...	编辑到了工作空间入口	打开具体功能包CMakeLists.txt
rospack报python-*依赖不存在	rosdep索引或环境/旧包依赖等	核对ROS_DISTRO、包路径、rosdep输出；不盲装Python2包
command not found或找不到自建包	环境未加载或编译未完成	source Noetic与devel/setup.bash，查看第一条构建错误
节点在运行但无消息	话题/类型不一致或未处理回调	rostopic info/type、rosnode info、检查spin和映射
mimic不控制目标乌龟	实际乌龟名与output不一致	对照rostopic list后改映射
退出连续pub	进程持续发送	在其终端Ctrl+C

“Invoking make failed”通常是结果，真正原因在前面第一条CMake/Error信息。排错路径从“运行环境→文件存在→XML语法→依赖→CMake顺序→代码→运行名称”依次定位；不能一遇错误就删除工作空间重建。

14.2 课件勘误与避免照抄

课件表述/写法	复习时采用的准确写法
std_msgs/Strings.h	std_msgs/String.h，String无复数s
turtlesim_teleop_key	turtle_teleop_key
#！/bin/bash或中文引号	#!/bin/bash，代码中半角标点
dpkg -e用于卸载	-e提取控制信息；卸载用-r或-P
/soft.bin执行当前文件	./soft.bin
chmod一律需要root	文件所有者通常可自行改权限
发布必须spinOnce	spinOnce处理回调；纯发布无需一概强制
person需求与定义类型不一致	遵循题干；本资料采用最终定义页
PPA源固定三个实际语法字段	课件三组讲法；真实源需区分类型URI套件组件
OPP	通常为OOP（面向对象编程）

课件旧安装地址/密钥不是本资料承诺可直接在线执行的当前安装方案；考试定义与当前运维步骤分别理解。本文中的程序/配置范例已校对字段、类型、名称与顺序，未把PDF生成检查写成Ubuntu运行验证。

14.3 五种典型答法示例

概念题“消息是什么”：消息是节点通信使用的数据结构/实例；其定义指定字段类型与名称；例如Twist包含linear与angular。

比较题“包与节点区别”：功能包是磁盘上的代码、清单和配置组织单位；节点是运行时计算单元；一个功能包可提供多个节点可执行程序。

流程题“自定义消息步骤”：建msg→配置package.xml→配置CMake→编译生成接口→在节点包含头文件使用类型→构建节点→source并运行。

代码题“Rate r(10)含义”：为循环设置10 Hz目标频率，sleep尽量补足周期；不是每10秒执行一次，也不是队列长度10。

改错题“XML根外依赖”：指出错误位置，解释只有一个根元素，再把依赖移至</package>之前；只说‘格式不对’不够完整。

附录A 已提供的7组填空题：116空速查

以下按本对话中图片的空号排列，供遮挡答案自测。每组空号重新从1计。术语沿用课件，歧义处同时给出解释。

A1 Linux文件系统，16空

空号	答案与提示
1-2	索引节点；数据
3-6	普通文件；目录文件；设备文件；链接文件
7-9	树形（目录树）；根目录；/
10-11	绝对路径；相对路径（两者次序按题干）
12	根目录/
13-14	/usr；/var
15	基本命令/基本可执行程序
16	附加或第三方软件包

A2 终端与基础命令，17空

空号	答案与提示
1-2	仿真终端；文本终端（TTY）
3	Ctrl+Alt+T
4-7	用户名；主机名；当前目录；提示符（权限身份标志）
8	以获授权的身份（通常root）执行命令
9-10	命令解释器；编程语言/脚本环境
11	Bash
12-15	ls；cd；mkdir；echo
16-17	cd或cd ~；cd ..

题目中的“当前目录清单”为ls，不是pwd；pwd显示路径。

A3 Shell与数据流，16空

空号	答案与提示
1-2	交互式；批处理式
3	指定脚本运行环境/解释器
4-6	直接执行；指定Shell执行；输入重定向到Shell
7-9	用户自定义变量；环境变量；系统变量
10	$
11	配置用户Bash环境，在相应Shell启动时加载设置
12	可执行命令的搜索路径
13-15	标准输入stdin；标准输出stdout；标准错误stderr
16	管道符号为竖线（见04章代码）

A4 归档压缩，11空

空号	答案与提示
1-3	tar；打包（归档）；解包（提取）
4-5	.gz；gunzip或gzip -d
6-7	.bz2；bunzip2或bzip2 -d
8-9	.zip；unzip
10	tar -zxvf file.tar.gz
11	tar -jxvf file.tar.bz2

A5 文件身份与权限，24空

空号	答案与提示
1-3	所有者；所属组；其他用户
4-6	u；g；o
7-9	读；写；执行
10-12	r；w；x
13-15	4；2；1
16	ls -l
17-19	文件类型；d；-
20-22	chown；chgrp；chmod
23	/home/ros/myfile的u/g/o均设为读、执行，无写权限
24	chmod u=rw,go=r file

A6 软件包管理，13空

空号	答案与提示
1-2	源码包；二进制包
3	DPKG（软件包管理体系/工具）
4-6	./configure；make；make install（需特权时sudo）
7-8	apt；dpkg
9	更新本地软件包索引/课件称更新软件源列表
10	升级已安装的软件包
11-13	软件包类型；源URL；发行版本或分类（课件分组讲法）

第11-13空来自传统APT源行讲解，真实语法还应分清发行套件和组件。PPA简写ppa:user/ppa-name本身不是上述三个空格字段。

A7 ROS入门与安装，19空

空号	答案与提示
1	提高机器人研发领域的代码复用率
2-5	通讯机制；开发工具；应用功能/高层技能；生态系统
6-12	配置Ubuntu仓库；添加ROS源；配置密钥；安装Noetic；环境设置；解决依赖/初始化更新rosdep；测试
13	sudo apt install ros-noetic-A（A为实际APT包名）
14	运行指定功能包中的可执行程序
15	跳转到功能包目录
16-19	turtlesim_node；turtle_teleop_key；draw_square；mimic

附录B 主观题的标准答题要点

B1 Linux与Shell类

问：终端、Shell与TTY区别联系？答：终端提供文本交互接口，仿真终端是图形窗口程序；Shell解释执行命令；TTY表示终端设备/文本控制台接口。用户通过终端操作Shell，由Shell调用程序并显示结果。

问：执行Shell脚本有哪些方法？答：赋予执行权限后用路径直接执行；显式指定bash/sh执行；把脚本内容重定向为Shell输入。source在当前Shell运行，用于环境加载，需说明与子Shell执行的差别。

问：命令<文件1>文件2表示什么？答：从文件1读取标准输入，执行命令，将标准输出覆盖写入文件2；错误输出不因单纯>而自动写入文件2。

问：归档作用？答：把多个文件/目录组织成一个归档便于备份、传输和保存属性；压缩是减少体积，两者可结合。

问：chmod、chgrp、chown区别？答：分别修改权限、所属组、所有者；说明各自操作对象，并给例子。ls -l首字符表示文件类型，随后九位才是传统权限。

问：Ubuntu安装软件方法？答：源码包编译安装、deb包APT/dpkg安装、可执行bin安装、免安装包，以及通过PPA等源取得软件。按题干要求列层级，避免把“deb”和“APT”说成完全无关的两种包格式。

B2 ROS、工作空间与消息服务类

问：什么是ROS，首要目标？答：Robot Operating System，是面向机器人的元操作系统/软件框架，提供通信、工具和可复用功能等；首要目标是提高机器人研发代码复用率。

问：工作空间三个目录与创建命令？答：src源码/包、build中间构建、devel开发结果和环境；mkdir -p ~/catkin_ws/src、cd ~/catkin_ws、catkin_make。

问：创建依赖std_msgs、rospy、roscpp的功能包a？答：先cd ~/catkin_ws/src，再catkin_create_pkg a std_msgs rospy roscpp；之后回工作空间根目录编译。

问：XML声明与元素特征？答：声明常写版本和编码；元素大小写敏感、匹配闭合、可嵌套不可交叉、只有一个根元素，属性值带引号。自闭合写法也合法。

问：Catkin最小结构？答：自己的包目录、符合规范的package.xml、Catkin版CMakeLists.txt。src/srv/msg/launch分别放源代码、服务定义、消息定义、启动配置。

问：消息字段与常量区别？答：字段是每个消息实例的可变数据，写类型和名称；常量是固定定义值，写类型名称=值。服务由请求和响应组成，以---分隔。

问：消息与服务作用？答：消息定义节点传输的数据；话题利用消息实现异步发布订阅；服务利用请求/响应结构实现同步调用与返回结果。不能把消息只说成一个运行话题。

B3 计算图与程序类

问：Master做什么？答：记录名称与注册信息，帮助节点发现通信对象、建立联系；实际话题数据通常由节点直接传输。参数服务器提供配置存取。

问：为什么订阅者需要spin？答：保持程序运行并处理回调队列，让收到的消息触发已注册回调；仅声明Subscriber不等于完成回调处理。

问：自定义msg的完整步骤？答：创建定义、声明生成/运行依赖、CMake登记并生成、编译获得接口、在节点包含头文件使用类型、构建节点、加载环境并运行检查。

问：如何理解launch中的pkg/type/name？答：包名定位程序所在包，type指定可执行文件，name指定运行节点名；三个名称在不同层级，不必相同。

答题时优先用题干要求的术语列要点，然后每项加一句作用。只有概念问题才需要解释，不要把完整程序抄到一个两分定义题里。

附录C 自拟期末模拟卷与参考答案

本卷用于训练，100分、建议90分钟。分值为练习设计，不代表学校试卷结构。先闭卷完成题目，再查看后面的答案。

C1 题目

一、填空题（20分，每空1分）

1. Linux目录树最上层为（1），符号为（2）。

2. 文件三类访问者的简写是（3）（4）（5）；三种权限数值是r=（6）、w=（7）、x=（8）。

3. Ubuntu常用Shell是（9）；打开仿真终端快捷键是（10）。

4. 创建嵌套目录时mkdir的选项为（11）；不覆盖已有文件的cp选项为（12）。

5. ROS首要目标是（13）；Noetic配套Ubuntu版本为（14）。

6. Catkin工作空间源码目录为（15）；开发结果目录为（16）。

7. 请求和响应在.srv中用（17）分隔；消息运行时依赖为（18）。

8. 节点初始化API为（19）；服务客户端发起同步调用的方法为（20）。

二、单选与判断（10分，每题2分）

9. 哪个命令查询运行话题的消息类型？A rosmsg show /chatter；B rostopic type /chatter；C rosservice list；D roscd chatter。

10. 哪个最适合连续发布位姿？A话题；B一次求和服务；C修改package.xml；D清理APT缓存。

11. chmod 640 file中，组用户权限为？A读写；B只读；C读执行；D无。

12. “ROS Master负责转发全部话题消息。”判断正误并用一句话解释。

13. “ROS_INFO能替代Publisher.publish发送消息。”判断正误并解释。

三、命令题（20分，每题4分）

14. 创建~/catkin_ws/src并在工作空间根目录编译，写出三条命令。

15. 将/etc复制到/tmp且不覆盖已有文件。

16. 把course目录打包压缩成course.tar.bz2，再解压该文件。

17. 将file设为所有者读写、组和其他只读，分别用数字法和符号法。

18. 查询/turtle1/cmd_vel的消息类型，再查看geometry_msgs/Twist字段。

四、简答题（20分，每题5分）

19. 区分ROS两套三层结构，分别列出名称。

20. 比较话题与服务：通信模式、数据定义、适用场景。

21. 区分package.xml与CMakeLists.txt，并说明修改自定义消息生成配置应在哪个CMake文件。

22. 写出自定义msg从定义到节点可使用的主要步骤。

五、配置与改错（15分）

23. 在包中定义name、age、height，分别为string、uint16、float64，写person.msg（3分）。

24. 给CMake顺序排序：A catkin_package；B find_package；C generate_messages；D add_message_files（4分）。

25. 解释并修正std_msgs/Strings.h、mkdir-p~/catkin_ws/src，以及将依赖写在</package>后的错误（各2分，共6分）。

26. 写一条node标签：运行turtlesim包的turtlesim_node，节点名sim，终端显示输出（2分）。

六、C++编程（15分）

27. 编写订阅/chatter的节点listener，消息类型std_msgs/String，回调打印data。要求含头文件、初始化、句柄、Subscriber、回调处理。说明编译该文件需增加的两条CMake命令。

C2 参考答案与给分点

填空1-20：根目录；/；u；g；o；4；2；1；Bash；Ctrl+Alt+T；-p；-n；提高机器人研发代码复用率；20.04；src；devel；---；message_runtime；ros::init；call（client.call）。

第9题B；第10题A；第11题B。第12题错误：Master注册/发现，节点通常直接传输数据。第13题错误：ROS_INFO输出日志，publish发送话题消息。

命令题答案如下；同义参数组合和合理等效写法可接受。

```
mkdir -p ~/catkin_ws/src
cd ~/catkin_ws
catkin_make

cp -rn /etc /tmp/

tar -jcvf course.tar.bz2 course/
tar -jxvf course.tar.bz2

chmod 644 file
chmod u=rw,go=r file

rostopic type /turtle1/cmd_vel
rosmsg show geometry_msgs/Twist
```

第19题：系统层/中间层/应用层，以及文件系统级/计算图级/开源社区级。名称正确各给分，不能把两组混成一组。

第20题：Topic发布订阅、异步、.msg、持续数据流；Service请求响应、通常同步、.srv请求/响应、一次操作和返回结果。至少在同一维度比较。

第21题：package.xml记录包元信息与阶段依赖；CMakeLists.txt配置构建/消息生成/目标链接等。应改src/具体包/CMakeLists.txt，不是工作空间src/CMakeLists.txt。

第22题：建msg→声明message_generation/message_runtime等依赖→登记与生成消息→编译→包含生成接口编写节点→配置目标、再次编译→source运行检查。写出主要依赖和生成顺序。

第23题按09章person.msg写三行。第24题B-D-C-A。第25题依次改String.h、mkdir -p ~/catkin_ws/src、把依赖移至package根元素内。第26题：

```
<node pkg="turtlesim" type="turtlesim_node"
      name="sim" output="screen" />
```

第27题参考：

```
#include <ros/ros.h>
#include <std_msgs/String.h>

void cb(const std_msgs::String::ConstPtr& msg) {
    ROS_INFO("%s", msg->data.c_str());
}
int main(int argc, char** argv) {
    ros::init(argc, argv, "listener");
    ros::NodeHandle nh;
    ros::Subscriber sub = nh.subscribe("/chatter", 1000, cb);
    ros::spin();
    return 0;
}
```

```
add_executable(listener src/listener.cpp)
target_link_libraries(listener ${catkin_LIBRARIES})
```

评分建议：头文件2；初始化/句柄3；回调签名、读data与日志3；subscribe3；spin2；两条CMake2。该题预设基础find_package、catkin_package和include_directories已配置；若是从零创建包，还应补清单和基础CMake。

错题回练：命令错误按路径与参数重写；概念错误做一对一对照卡；程序错误按角色和数据流重写骨架。不要只在答案上划线后认为已经掌握。

附录D 覆盖核对、后续专题与考前清单

D1 用章末题检查自己是否遗漏

课件章末题/重点	已覆盖内容
Lesson1 p25、60、98	ROS四位一体；GNU/Linux/Ubuntu；虚拟机与桌面
Lesson2 p27、75	文件类型/FHS；终端ShellTTY；五道基础命令
Lesson3 p24、53、71	脚本/重定向管道；tar压缩；身份权限与三条变更命令
Lesson4 p21	源码、deb、APT、PPA与安装流程
Lesson5 p25、54	定义/目标；Noetic、软件源、bashrc、rosdep、rosbash
Lesson6 p22、57、83	两套结构、空间与包；XML/清单；字段常量与服务
Lesson7学习目标/总结	CMake生成顺序、计算图工具、turtlesim与mimic
Lesson8学习目标/总结	C++编译调试、类、命名空间、Makefile、launch标签
Lesson9 p15、32、63	11步骤、发布订阅与通信模型
Lesson10 p14-40	自定义消息和服务、6个配套程序

所有已提供图片中的7组填空共116空已在附录A给出；可见主观题归入知识章节与附录B；实际报错与turtlesim操作问题在10、14章。未取得完整题干的40道选择题不能作逐题答案核验。

D2 总纲后续专题：需要另补的资料

以下来自Lesson1课程安排所列范围，细节未在这10份课件展开。这里只提供通用基础概念作定位，不能据此认定已经覆盖完整专题考试。

专题	基础概念	若老师授课/考试涉及，应补什么
TF2	管理带时间的坐标系变换	变换方向、坐标帧关系、广播/查询API与计算题
URDF	XML描述机器人连杆、关节、视觉/碰撞/惯性等	link、joint、origin、轴向与机器人模型例子
Xacro	通过宏和参数简化URDF等XML	宏定义/调用、属性、生成URDF步骤
Rviz	可视化ROS数据与机器人状态	Fixed Frame、显示项、TF与消息配置
Gazebo	机器人/物理环境仿真	模型、传感器、插件与ROS连接
SLAM	同时定位与建图	传感器输入、地图、算法/包和启动配置

专题	基础概念	若老师授课/考试涉及，应补什么
地图服务	提供地图数据	map_server、地图文件与话题
定位与控制	估计位姿并驱动运动	定位输入、坐标系、速度命令和误差
路径规划	由起点/目标生成路径或运动指令	全局/局部规划、代价地图与导航流程

若期末范围只到ROS基础，本资料主体可直接用于复习；若总纲后续内容也在范围内，需加入相应课件、课堂例题和教师要求，不能仅用概念表替代。

D3 考前闭卷检查

- 能区分文件系统路径、工作空间、功能包、节点、话题、消息类型和服务名。
- 能写基础十条命令及常用参数，尤其mkdir -p、cp -rn和路径切换。
- 能解释三个数据流、重定向和管道，写四条tar命令。
- 能将权限字符串与数字互换，区分chmod/chown/chgrp。
- 能解释APT update/upgrade、源码安装三步、PPA源。
- 能默写ROS目标、四位一体、两套三层结构与安装七步。
- 能写工作空间三条命令、创建包命令，解释source作用。
- 能检查XML根、标签和属性，写六种依赖标签。
- 能写.msg/.srv，明确字段/常量、请求/响应。
- 能给CMake关键步骤排序，知道生成头文件和目标依赖。
- 能讲清发布订阅和服务调用，理解Master不转发业务数据。
- 能运行/观察turtlesim，清屏、生成第二只、正确映射mimic。
- 能解释launch各属性与常见标签，并写一个完整launch。
- 能手写String发布者/订阅者和求和服务端/客户端的骨架。
- 能根据第一条错误定位问题，而不是只重复make失败结果。

D4 来源与使用说明

主要来源为D:/Personal/Desktop/课程PPT中的Lesson1.pdf至Lesson10.pdf，文件页数依次102、79、75、272、58、87、84、82、65、426。每章列出主要定位页；重复复习页已合并。

本资料结合本对话已提供的Linux、Shell、归档、权限、软件管理、ROS安装、XML、消息服务、计算图、turtlesim、roslaunch题图与命令报错。未采用附件内容作为额外执行指令。

补充例题与C++代码用于将课件知识连成可复习流程；模拟卷为自拟。涉及课件不准确或前后不一致之处，以14章勘误及具体题干要求为准。复习优先级是资料整理判断，期末最终范围仍以教师公布内容为准。